



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Carandia Lorenzo Brenda Fernanda

N° de Cuenta: 319018961

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 29 de marzo de 2025

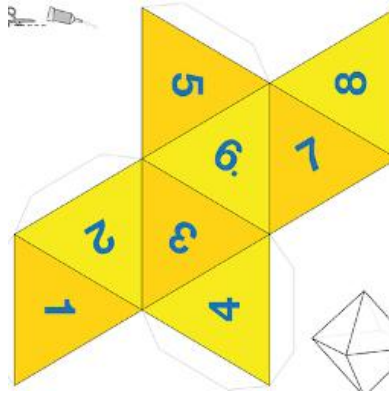
CALIFICACIÓN: _____

TEXTURIZADO

Actividades Realizadas

- **Ejercicio 1:** Crear un dado de 8 caras y texturizarlo por medio de código.

Imagen modificada para nuestra textura



256x256

Código

```
void crearOctaedro()
{
    unsigned int octaedro_indices[] = {
        0, 1, 2, // Cara inferior 1
        3, 4, 5, // Cara inferior 2
        6, 7, 8, // Cara inferior 3
        9, 10, 11, // Cara inferior 4
        12, 13, 14, // Cara superior 1
        15, 16, 17, // Cara superior 2
        18, 19, 20, // Cara superior 3
        21, 22, 23 // Cara superior 4
    };

    // Asignar coordenadas de textura específicas para cada vértice en cada cara
    GLfloat octa_vertices_completos[] = {
        // Cara inferior 1
        //x y z S T NX NY NZ
        0.0f, -1.0f, 0.0f, 0.6815, 0.7876, 0.0f, -1.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.9948, 0.9735, 0.0f, 0.0f, 1.0f,
        1.0f, 0.0f, 0.0f, 0.9952, 0.5995, 1.0f, 0.0f, 0.0f,

        // Cara inferior 2
        //x y z S T NX NY NZ
        0.0f, -1.0f, 0.0f, 0.6739, 0.7841, 0.0f, -1.0f, 0.0f,
        1.0f, 0.0f, 0.0f, 0.9981, 0.5938, 1.0f, 0.0f, 0.0f,
        0.0f, 0.0f, -1.0f, 0.6746, 0.4134, 0.0f, 0.0f, -1.0f,

        // Cara inferior 3
        //x y z S T NX NY NZ
        0.0f, -1.0f, 0.0f, 0.6675, 0.7797, 0.0f, -1.0f, 0.0f,
        0.0f, 0.0f, -1.0f, 0.6673, 0.4064, 0.0f, 0.0f, -1.0f,
        -1.0f, 0.0f, 0.0f, 0.3539, 0.5920, -1.0f, 0.0f, 0.0f,

        // Cara inferior 4
        //x y z S T NX NY NZ
        0.0f, -1.0f, 0.0f, 0.6556, 0.7885, 0.0f, -1.0f, 0.0f,
        -1.0f, 0.0f, 0.0f, 0.3503, 0.6052, -1.0f, 0.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.3512, 0.9722, 0.0f, 0.0f, 1.0f,

        // Cara superior 1
        //x y z S T NX NY NZ
        0.0f, 1.0f, 0.0f, 0.3335, 0.2089, 0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.8218, 0.3901, 0.0f, 0.0f, 1.0f,
        1.0f, 0.0f, 0.0f, 0.8256, 0.8224, 1.0f, 0.0f, 0.0f,

        // Cara superior 2
        //x y z S T NX NY NZ
        0.0f, 1.0f, 0.0f, 0.3578, 0.2087, 0.0f, 1.0f, 0.0f,
        1.0f, 0.0f, 0.0f, 0.6672, 0.9198, 1.0f, 0.0f, 0.0f,
        0.0f, 0.0f, -1.0f, 0.6677, 0.3932, 0.0f, 0.0f, -1.0f,

        // Cara superior 3
        //x y z S T NX NY NZ
        0.0f, 1.0f, 0.0f, 0.3522, 0.2212, 0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, -1.0f, 0.6569, 0.4066, 0.0f, 0.0f, -1.0f,
        -1.0f, 0.0f, 0.0f, 0.3521, 0.5809, -1.0f, 0.0f, 0.0f,

        // Cara superior 4
        //x y z S T NX NY NZ
        0.0f, 1.0f, 0.0f, 0.3389, 0.2168, 0.0f, 1.0f, 0.0f,
        -1.0f, 0.0f, 0.0f, 0.3379, 0.5822, -1.0f, 0.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.8327, 0.3984, 0.0f, 0.0f, 1.0f
    };

    Mesh* octaedro = new Mesh();
    octaedro->CreateMesh(octa_vertices_completos, octaedro_indices, 24 * 8, 24);
    meshList.push_back(octaedro);
}
```

Creamos nuestro octaedro, en donde listamos cada cara de la parte superior como inferior, además le pasamos las coordenadas de la imagen que queremos observar en la cara.

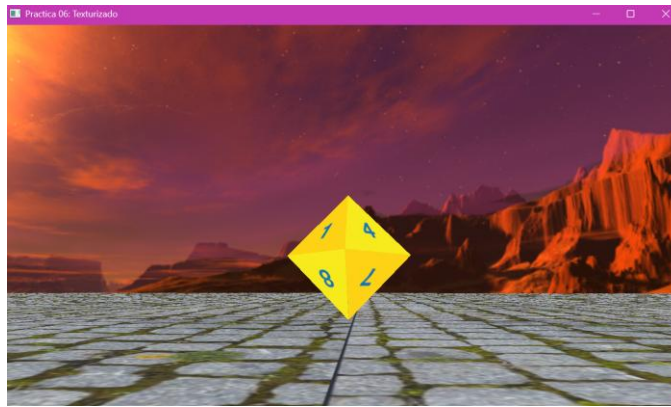
```
datoTexture = Texture("Textures/octaedro.jpg");  
datoTexture.LoadTextureA();
```

Le pasamos la dirección de donde se encuentra nuestra imagen

```
//Dado de OpenGL  
//Ejercicio 1: Texturizar su cubo con la imagen dado_animales ya optimizada por ustedes  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
datoTexture.UseTexture();  
meshList[4] -> RenderMesh();
```

Lo mandamos a llamar en el main.

Ejecución



- **Ejercicio 2:** Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin)
- **Ejercicio 3:** Texturizar la cara del personaje de la imagen tipo cars en el espejo (ojos) y detalles en cofre y parrilla de su propio modelo de coche



Coche de referencia



Utilice la imagen de arriba para texturizar al carro en 3D Max

```
//carro importado
model = glm::mat4(1.0);
//model = glm::translate(model, glm::vec3(-3.0f, 3.0f, -2.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.05f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro_texturizado.RenderModel();
```

Lo importamos en el código

Ejecución del código



- Problemas presentados.

En esta práctica no se me presentaron problemas a la hora de ejecutar el código.

- Conclusiones

Estos ejercicios fueron de gran ayuda para entender cómo funciona la texturización en un modelo, lo que más me costó trabajo fue entender un poco el texturizado en el código, me confundían las coordenadas, pero pude entender. Sin duda la complejidad fue la adecuada.

Bibliografía

- Corbalán Navarro, D. (2023). *Técnicas de texturizado en OpenGL*. Recuperado de https://digitum.um.es/digitum/bitstream/10201/132724/1/Corbal%C3%A1n-Navarro-David_TD_2023.pdf